

CSCE 608 DATABASE SYSTEMS · MODULE 02 · LECTURE NOTES

Storage and the Buffer Pool

Pages, and deciding what stays in memory.

MODULE OUTCOMES

- Explain why databases are organised in fixed-size pages.
- Design a slotted page layout for variable-length records.
- Implement a buffer pool with pin counts and dirty tracking.
- Compare replacement policies and explain why LRU fails on scans.
- Explain why a DBMS bypasses the OS page cache.

1 Pages

THE STORAGE HIERARCHY DICTATES THE ALGORITHMS

Approximate random-access latencies: registers under a nanosecond; RAM around 100 ns; NVMe SSD around 100 µs; a spinning disk around 10 ms.

Memory to SSD is a factor of a thousand; memory to disk is a factor of a hundred thousand. No clever algorithm overcomes a gap of that size — you design around it, which means the algorithm's goal is to minimise the *number of I/Os* rather than the number of operations.

And sequential access beats random access by ten to a hundred times even on SSDs, because of prefetching, request coalescing, and queue depth. On spinning disks the ratio is far larger still.

Three consequences, and they explain nearly everything: read in large fixed-size blocks rather than individual records; keep as much as possible in memory; and prefer sequential access patterns even at the cost of reading more data than you need. A B+tree (Module 03) is what you get when you take these seriously.

Why fixed-size pages	Detail
Matches the hardware	Storage devices transfer blocks, not bytes. Reading one byte and reading four kilobytes cost the same.
Simple allocation	Uniform size means a page file never fragments — any free page can hold any page's contents.
Directly addressable	A page ID multiplied by the page size is a file offset. No lookup table is needed to find a page.

Why fixed-size pages	Detail
The unit of I/O and of caching	The buffer pool deals exclusively in pages, which keeps it simple.
The unit of recovery	Log records reference pages and their LSNs (Module 11).

4 KB to 16 KB is typical. A larger page reads more useful data per seek, which helps scans, and wastes more of the transfer when only one record is wanted, which hurts point lookups. The choice therefore depends on the workload, and this trade recurs throughout the course.

1.1 The slotted page

```

+-----+
| HEADER | slot0 | slot1 | slot2 | -->   free space   <-- |
+-----+
|                                     | rec2 | rec1 | rec0 |
+-----+

slot array grows FORWARD    records grow BACKWARD
page is full when they meet
each slot = (offset, length); record address = (page_id, slot_no)

```

THE SLOT ARRAY EXISTS SO RECORDS CAN MOVE

The design looks needlessly indirect until you ask what happens when a record is deleted and the resulting hole must be reclaimed.

With a slot array, a record can be relocated anywhere within its page — to compact free space — without changing its identity. Its address is (page, slot number), and only the slot's offset field changes.

Which means every index entry pointing at that record remains valid. Without the indirection, compacting a page would require finding and updating every index entry referencing every record that moved — which would make compaction impossible in practice.

That indirection is the entire point of the design, and it is the answer to 'why not just store records end to end'.

Complication	Handling
Variable-length fields	Store all fixed-length fields first at known offsets, then an array of offsets to the variable-length ones. A field's position is then either a constant or one indirection.
NULLs	A bitmap in the record header. Using a sentinel value instead is wrong and eventually collides with a legitimate value — the classic case being a date of 1900-01-01 meaning 'unknown'.
Records larger than a page	Overflow pages chained from the main record, or out-of-line storage for large attributes. PostgreSQL's TOAST mechanism compresses and then, if still too large, stores out of line — transparently.

Complication	Handling
An update that grows a record	Either compact the page to make room, or move the record to another page and leave a forwarding pointer in the original slot. Both approaches are used; forwarding pointers accumulate over time and turn one-page lookups into two, which is a significant reason tables need periodic reorganisation.
Deletes	Mark the slot free. Actual reclamation is deferred, and under MVCC the record cannot be removed at all until no transaction can still see it (Module 10).

2 The buffer pool

The buffer pool is the boundary between the database and the disk. It holds a fixed number of page-sized frames, a mapping from page IDs to frames, and the policy for deciding which page to evict when a frame is needed.

```
struct Frame {
    page_id_t page_id;
    char      data[PAGE_SIZE];
    int       pin_count;    // > 0: in use, NEVER evict
    bool      is_dirty;     // modified; must be written before eviction
};
```

PIN COUNTS ARE THE INVARIANT THAT MATTERS

A pinned page is one that some caller currently holds a pointer into. Evicting it and loading a different page into that frame would leave the caller reading another page's bytes — and the corruption is silent, because the data looks like perfectly valid data of the wrong page.

Every fetch must be matched by exactly one unpin. A leaked pin permanently removes a frame from circulation; enough leaks and the pool has nothing evictable and every subsequent fetch fails.

Use RAIL, or your language's equivalent. Unpinning manually on every control path — including every error path and every early return — is not something anyone achieves reliably, and the bug is invisible until the pool is under pressure.

Handle the all-pinned case explicitly. Returning a failure when every frame is pinned is correct behaviour, not an impossible condition, and omitting it turns a recoverable situation into a crash or a corruption.

3 Replacement policies

Policy	Mechanism	Assessment
LRU	Evict the page not accessed for longest.	Intuitive and fails badly on sequential scans — see below.
CLOCK	Approximate LRU: frames in a circular buffer, each with a reference bit. Sweep a hand, clear set bits, evict the first unset one.	Much cheaper than true LRU (no list manipulation on every access) and nearly as effective. Very widely used.
LRU-K	Track the time of the K-th most recent access, and evict by that rather than by the most recent.	Resists scan pollution , because a page accessed only once has no K-th access and is evicted first. LRU-2 is the usual choice : one access is not evidence of value; two is.
2Q, ARC	Separate queues for pages seen once and pages seen repeatedly, with adaptive sizing.	Effective, with more bookkeeping. ARC's patent history pushed many systems toward alternatives.
MRU	Evict the <i>most</i> recently used page.	Sounds perverse and is exactly right for a sequential scan : the page just read will not be needed again, while older pages might. A good illustration that the correct policy depends on the access pattern, which is why some systems let the executor hint.

SEQUENTIAL FLOODING

A full table scan reads every page of a large table exactly once, in order.

LRU dutifully caches all of them. Each newly read page is the most recently used, so it is the last candidate for eviction — and the pages being evicted to make room are the hot working set that thousands of other queries depend on.

The result is that one analytical query destroys the cache for every transactional query on the system, in exchange for caching pages that will never be read again. Performance collapses, and recovers only slowly as the working set is faulted back in.

LRU-K fixes it directly. A page needs K accesses before it is considered valuable; a scan touches each page once, so the scanned pages are the first evicted and the working set is untouched. This is the main reason real systems do not use plain LRU.

4 Why not let the operating system do it?

The operating system already has a page cache and mmap already maps files into memory. The question of why a database reimplements all of this is a reasonable one, and the answer is instructive.

Problem with mmap	Consequence
No control over eviction	The OS does not know which pages matter for this workload, and it knows nothing about pin counts. It can evict a page the DBMS is actively using.
No control over write ordering	Decisive. The write-ahead logging protocol (Module 11) requires that specific writes reach durable storage before specific others. With mmap, the OS writes dirty pages whenever it chooses, in whatever order. This is not an inconvenience — it makes durability unachievable.
Unpredictable stalls	A page fault blocks the faulting thread inside the kernel. The DBMS cannot schedule other work, cannot issue the I/O asynchronously, and cannot even tell that it is waiting.
Interference from OS features	Transparent huge pages, readahead heuristics, and NUMA migration all act on mapped memory in ways that cannot be disabled per file.
Double buffering	If the DBMS has its own cache <i>and</i> the OS page cache is active, hot pages occupy memory twice.

Crotty, Leis and Pavlo's paper *Are You Sure You Want to Use MMAP in Your DBMS?* makes this case in full; the title is effectively the abstract. Systems that tried it — MongoDB's original storage engine, among others — moved away.

O_DIRECT, FSYNC, AND A TWENTY-YEAR BUG

O_DIRECT bypasses the OS page cache, so the DBMS's buffer pool is the only cache in the system. This eliminates double buffering and makes behaviour predictable, at the cost of losing the OS's readahead — which the DBMS must then implement itself, and can do better, because it knows what it is about to read.

fsync is how a program learns that a write has reached durable storage. It is expensive, which is why the log batches writes and amortises the cost across transactions (Module 11).

And fsync can fail. Worse: on Linux and several other systems, a failed fsync historically marked the error as *consumed*, so a subsequent retry returned success while the data was gone. A program that retried on failure would conclude it had succeeded.

This is 'fsyncgate'. It affected PostgreSQL for roughly twenty years before being diagnosed in 2018, and the eventual resolution was to treat any fsync failure as unrecoverable and deliberately crash, so that recovery runs from the log. **Durability is substantially harder than it appears**, and this is the clearest illustration of why.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lectures 3–6, Storage and Buffer Pools	Page layout, slotted pages, and the buffer pool with replacement policies. The primary source for this module.
Crotty, Leis & Pavlo — Are You Sure You Want to Use MMAP in Your DBMS? (free)	The §4 argument in full, with measurements.
O'Neil, O'Neil & Weikum — The LRU-K Page Replacement Algorithm (free)	The original LRU-K paper, including the sequential flooding analysis.
PostgreSQL fsync reliability discussion ('fsyncgate')	The §4 story, documented by the people who found it. Worth reading as a case study in how durability assumptions fail.
Database Internals — chapters on storage engines	Page layouts and buffer management from a different angle.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Measure the latency of random and sequential reads on your own machine, at several block sizes. Plot it and compare against the figures in §1.
- 2 Implement a slotted page supporting insert, delete, and retrieval by slot number. Verify that compaction does not change any record's address.
- 3 Add variable-length records and a NULL bitmap. Store a record containing a legitimate value that a sentinel scheme would have misread.
- 4 Implement a buffer pool with pin counts and dirty tracking. Write a test that fails if a pin is leaked.
- 5 Implement LRU and CLOCK replacement. Compare hit rates on a Zipfian workload at several pool sizes.
- 6 **Demonstrate sequential flooding:** run a hot working set, then issue one full scan, and plot the hit rate before, during, and after.
- 7 Implement LRU-2 and repeat. The hit rate should barely move during the scan.
- 8 Produce the Project 1 hit-rate-against-pool-size plot and explain the shape of the curve — in particular where it flattens and why.
- 9 Measure the cost of fsync on your machine, and the throughput difference between one fsync per write and one per hundred writes.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the approximate latency of a random read from RAM, SSD, and disk, and the three design consequences.
- 2 Give five reasons databases use fixed-size pages.
- 3 Draw the slotted page layout and explain why the slot array exists.
- 4 How are variable-length fields, NULLs, and oversized records handled?
- 5 What is a pin count, what breaks without it, and why use RAII?
- 6 Compare five replacement policies.
- 7 Explain sequential flooding and how LRU-K prevents it.
- 8 Give the decisive reason a DBMS does not use mmap.
- 9 What was fsyncgate and what was the resolution?